Классы функционалов, скорости сходимости

Классы функционалов

Определение: функционал $f : \mathbb{R}^n \to \mathbb{R}$ называется выпуклым, если

$$\forall x, y \in \mathbb{R}^n, \theta \in [0,1] \Longrightarrow f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta) f(y)$$

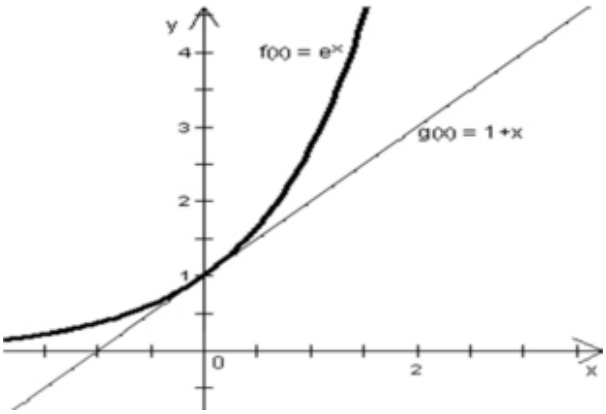
Определение: функционал $f : \mathbb{R}^n \to \mathbb{R}$ называется μ-сильно выпуклым, если

$$\forall x, y \in \mathbb{R}^n, \theta \in [0,1] \Longrightarrow f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta) f(y) - \mu\,\theta(1-\theta)\|x-y\|^2$$

Утверждение: Пусть $f(x) \in C^2(\mathbb{R}^n)$, тогда $f(x)$ --- выпуклый функционал $\iff \nabla^2 f \geq 0$

Если функционал $f(x)$ выпуклый, то его надграфик $\{(x,y) \in \mathbb{R}^{n+1} : y \geq f(x)\}$ лежит выше любой касательной к f.

Пример:



Утверждение: Пусть $f(x) \in C^2(\mathbb{R}^n)$, тогда $f(x)$ --- μ-сильно выпуклый функционал $\iff \nabla^2 f \geq \mu I$

Скорость сходимости

Рассмотрим невязку $r_n = \|x_n - x_*\|$ или $r_n = f(x_n) - f(x_*)$ или $r_n = \|\nabla f(x_n)\|$.

Определение: Последовательность r_n сходится линейно, если $\exists \rho \in (0,1) : r_n = O(\rho^n)$.

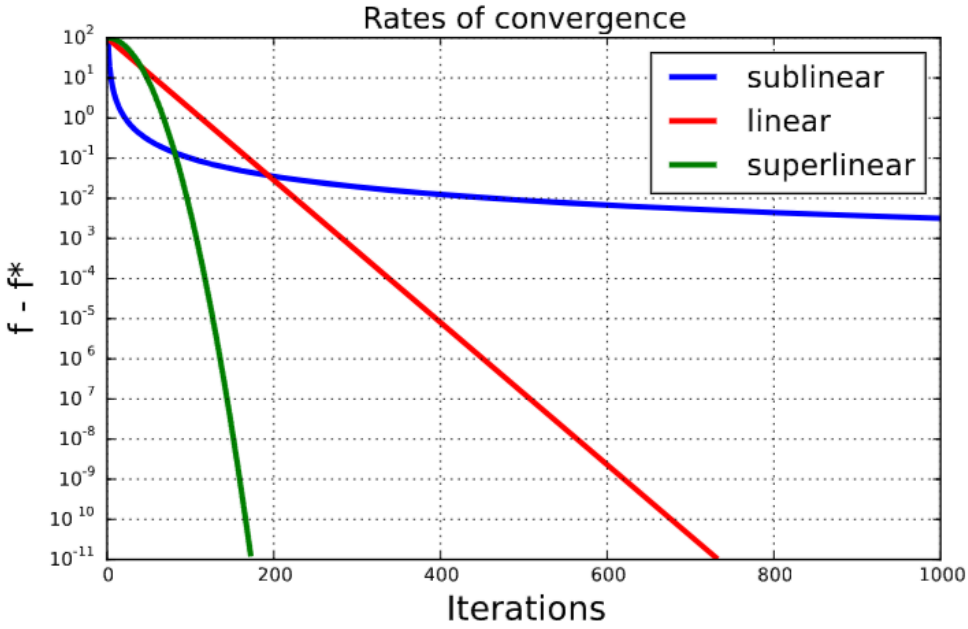
Линейная скорость сходимости означает, что для получения каждого знака нужно сделать нужно сделать константное число шагов.

Определение: Последовательность x_n сходится сублинейно, если $\nexists \rho \in (0,1) : r_n = O(\rho^n)$

Определение: Последовательность x_n сходится сверхлинейно, если
$\forall \rho \in (0,1) : r_n = O(\rho^n)$

Если метод сходится сублинейно, то для получения каждого следующего знака необходимо **больше** итераций. Таким образом, методы с сублинейной скоростью сходимости не позволяют получить высокую точность решения.

Если метод сходится сверхлинейно, то для получения каждого следующего знака необходимо **меньше** итераций. Таким образом, методы с сверхлинейной скоростью сходимости позволяют получать решение с высокой точностью.



Итерационные схемы, теорема Брауэра

Вам уже знаком вид градиентного спуска $x_{n+1} = x_n - \alpha \nabla f(x_n)$. В данном разделе будет построено обобщение данной схемы, также будут даны условия сходимости.

Рассмотрим процесс решения уравнения $\nabla f(x) = 0$. Пусть итерационная последовательность задается как $x_{n+1} = s(x_n)$, где $s(x)$ --- некоторый "хороший" (понятие "хороший" будет раскрыто ниже) оператор. Если итерационная последовательность сходится к некоторой точке x_*, то перейдя к пределу в равенстве $x_{n+1} = s(x_n)$ получим, что $x_* = s(x_*)$. Таким образом, x_* --- неподвижная точка $s(x)$. Мы хотим, чтобы точка x_* была стационарной $\nabla f(x_*) = 0$. Попробуем построить $s(x)$ так, чтобы все её неподвижные точки были решениями уравнения $\nabla f(x) = 0$.
Если $\nabla f(x_*) = 0$, то для любой невырожденной матрицы $B \Longrightarrow B\,\nabla f(x_*) = 0$. Хотим получить в правой части x_*, поэтому просто прибавляем к левой и правой частям x_*:
$x_* + B\,\nabla f(x_*) = x_*$. Для удобства поменяем знак перед вторым слагаемым и получим итоговую формулу:

$$x_* - B\,\nabla f(x_*) = x_*$$

Положим $s(x) = x - B\,\nabla f(x)$. Тогда её неподвижные точки --- решение уравнения $\nabla f(x) = 0$. Осталось понять, будет ли оператор $s(x)$ "хорошим" и будет ли сходиться итерационная последовательность $x_{n+1} = s(x_n)$? Для ответа на данные вопросы обратимся к теореме Брауэра.

Теорема (Брауэра о неподвижной точке): Пусть $s(x) \in C(\mathbb{R})$ и $\exists \rho \in (0,1):\ \forall x, y \in \mathbb{R}^n \Longrightarrow \|s(x) - s(y)\| \leq \rho \|x - y\|$ (отображение сжимающее, т.е. образы любой пары точек будут ближе, чем сами точки). Тогда $\exists!$ неподвижная точка x_*. Причем для любого начального приближения x_0 итерационная последовательность $x_{n+1} = s(x_n)$ сходится к x_* с линейной скоростью.

В ряде случаев мы не хотим или не можем проверить, будет ли $s(x)$ сжимающим. Попробуем упростить проверку данного условия.

Определение: Будем называть направление d направлением спуска $f(x)$ в точке x, если для достаточно малых $\lambda > 0$ будет выполнено $f(x + \lambda d) \leq f(x)$

Для $f(x) \in C^1(\mathbb{R}^n)$ данное условие эквивалентно $\langle \nabla f(x), d \rangle \geq 0$. Мы хотим, чтобы направление $d = B \nabla f(x_*)$ было направлением спуска:

$$\langle B\,\nabla f(x_*), \nabla f(x_*) \rangle \geq 0 \iff B \geq 0$$

В силу данного условия и невырожденности B далее будем требовать $B > 0$. В разделе "Демпфированный метод Ньютона, line search" мы рассмотрим методы выбора темпа обучения (скалярного множителя перед B), которые будут обеспечивать сходимость метода при движении по любым направлениям спуска (т.е. для любого выбора $B > 0$).

Таким образом, мы сформулировали общий вид итерационной схемы

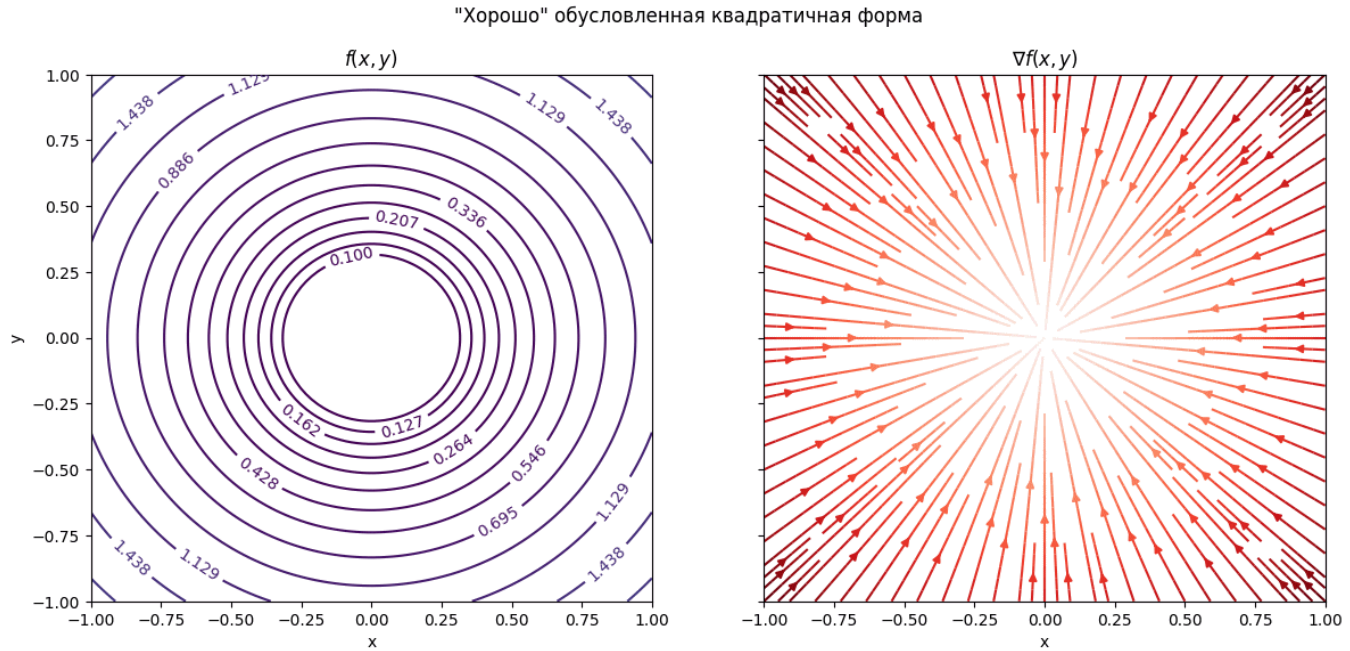
$$x_{n+1} = x_n - B(x_n) \nabla f(x_n), \quad B(x_n) > 0$$

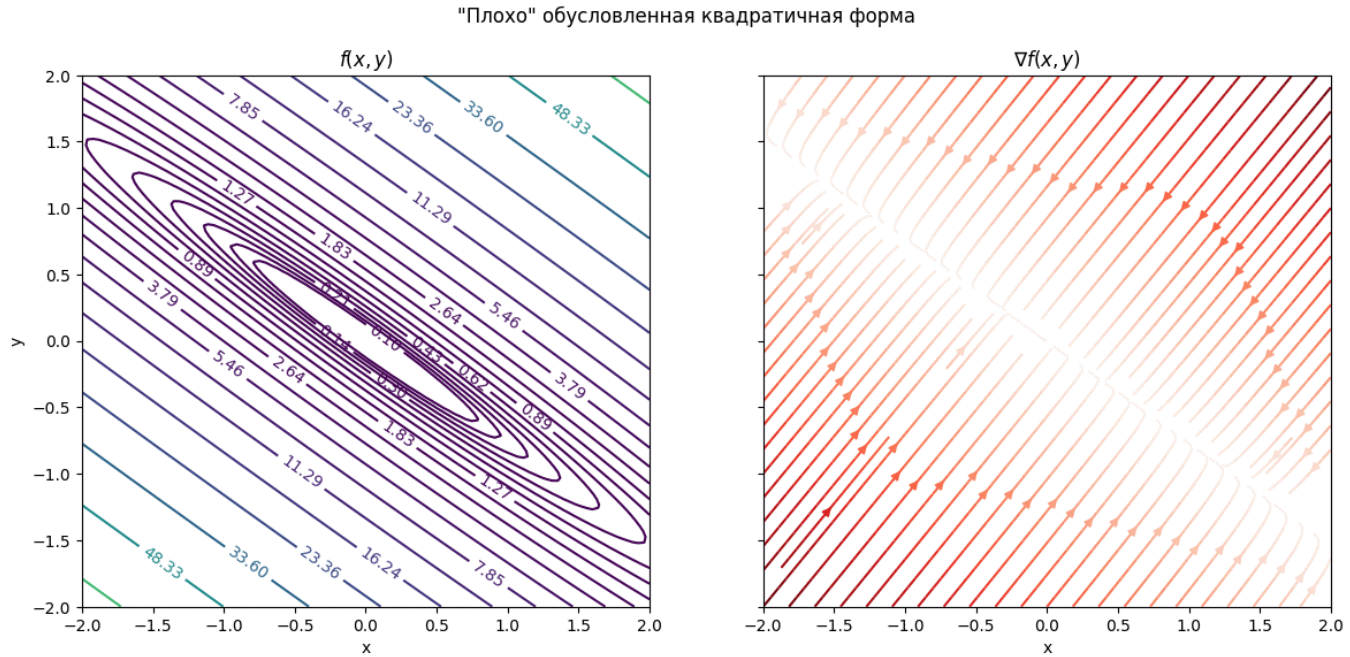
Проблемы градиентного спуска, предобуславливание градиентов, переход к новым координатам

В прошлом разделе был сформулирован общий вид итерационной схемы. В данном разделе мы обсудим выбор матрицы B.

Если положить $B = \alpha I$, то мы получим итерационную схему $x_{n+1} = x_n - \alpha \nabla f(x_n)$. Получается метод градиентного спуска. В данном методе мы шагаем вдоль векторного поля $\nabla f(x)$. На рисунках ниже показан ландшафт оптимизируемой функции $f(x)$,

векторное поле $\nabla f(x)$ и траектории оптимизации.





Можно заметить, что линии векторного поля $\nabla f(x)$ (изоклины) у "плохо обусловленных" функций $f(x)$ плохо спрямляются. Поэтому метод "сильно осциллирует" вдоль линий поля.

Мы хотим двигаться строго по изоклинам. Это означает, что мы хотим ввести (вообще говоря, криволинейные координаты), которые будут "двигаться вдоль изоклин". Такая замена координат существует (теорема о выпрямлении векторного поля), но поиск данной замены является не менее трудоемкой задачей, чем решение задачи оптимизации. Поэтому вместо поиска глобальной криволинейной координат построим для каждой точки свою декартову систему координат.

Пусть $x = Py$, обозначим $g(y) = f(Py)$. Для функции $g(y)$ градиент и гессиан имеют вид:

$$\nabla g(y) = P^T \nabla f(Py)$$
$$\nabla^2 g(y) = P^T \nabla^2 f(Py)\, P$$

Мы хотим "выпрямить" $\nabla g(y)$ в окрестности y_0. Под "выпрямлением" $\nabla g(y)$ мы будем понимать:

$$\nabla g(y) \approx \nabla g(y_0) + (y - y_0)$$

Разложим $\nabla g(y)$ в полином Тейлора в окрестности y_0:

$$\nabla g(y) = \nabla g(y_0) + \nabla^2 g(y_0)\,(y - y_0) + \overline{o}(\|y - y_0\|)$$

Отсюда получим:

$$\nabla^2 g(y) = P^T \nabla^2 f(Py)\, P \approx I$$

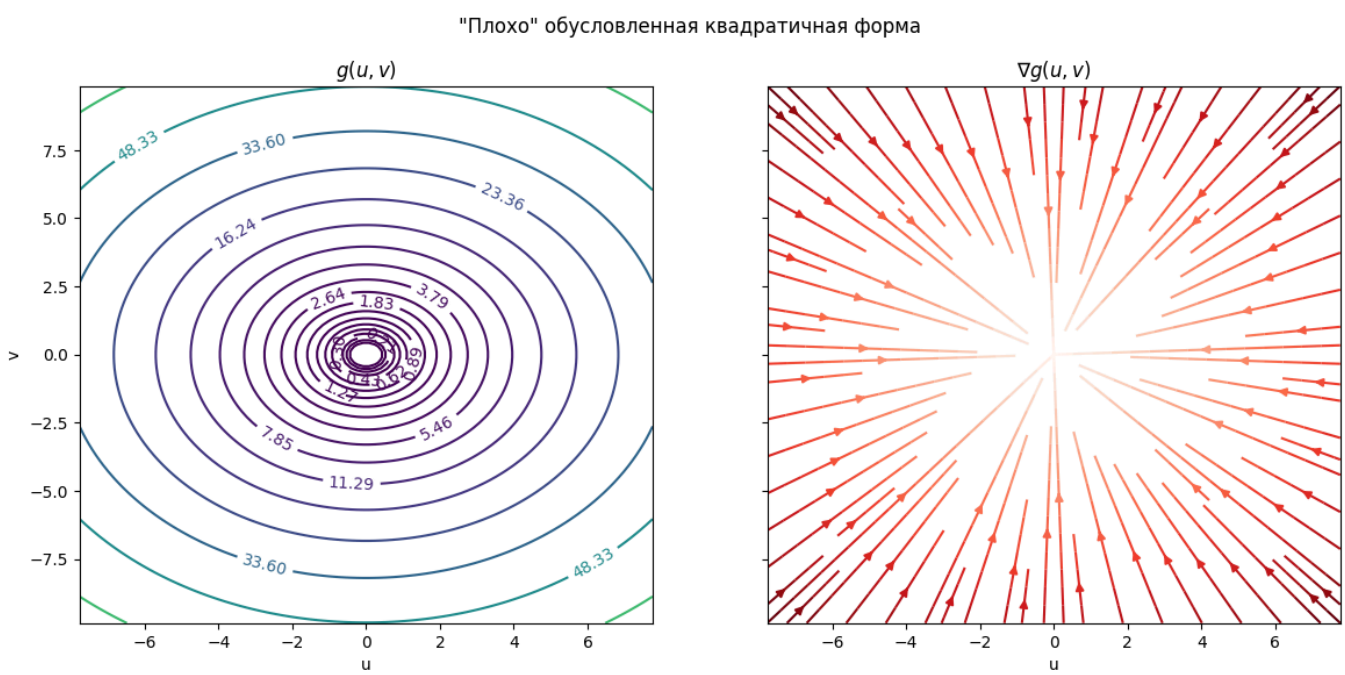
Чем лучше $\nabla^2 g(y)$ приближает единичную матрицу I, тем лучше "выпрямляется" $\nabla g(y)$

Попробуем обеспечить точное равенство $\nabla^2 g(y) = I$. Если $\nabla^2 f(Py) > 0$ (функция f сильно выпукла), то $\exists H > 0 : \nabla^2 f(Py) = HH^T$. Тогда:

$$P^T HH^T P = I$$

Данное равенство выполняется, если положить $P = H^{-1}$. Это равносильно $PP^T = (\nabla^2 f(Py_0))^{-1}$

На рисунках ниже изображены линии уровня $g(y)$ и линии векторного поля $\nabla g(x)$ (изоклины)



Сделаем шаг градиентного спуска для $g(y)$:

$$
\begin{aligned}
& y_{n+1} = y_n - \alpha \nabla g(y_n) \\
& \text{Обратная замена: } x = Py, \; y = P^{-1}x, \; \nabla g(y) = P^T \nabla f(x) \\
& P^{-1} x_{n+1} = P^{-1} x_n - \alpha P^T \nabla f(x_n) \\
& x_{n+1} = x_n - \alpha P P^T \nabla f(x_n) \\
& x_{n+1} = x_n - (\nabla^2 f(P y_0))^{-1} \nabla f(x_n)
\end{aligned}
$$

Полученную схему назовем методом Ньютона:

$$x_{n+1} = x_n - \alpha (\nabla^2 f(P y_0))^{-1} \nabla f(x_n)$$

В следующих разделах подробно изучим полученный метод.

Переход к новым (вообще говоря, криволинейным) координатам с целью "улучшения" свойств решаемой задачи называется предобуславливанием. Помимо продемонстрированного перехода $PP^T = (\nabla^2 f(P y_0))^{-1}$ распространены и другие замены координат.

Какую бы матрицу перехода P мы не выбрали, матрица PP^T будет положительно определена. Поэтому мы можем переходить к любой системе координат. Сходимость метода при правильном выборе α обеспечивает теорема Брауэра. Несколько других способов выбора PP^T будут рассмотрены в разделе, посвященном квазиньютоновским методам.

Классический метод Ньютона

В предыдущем разделе была выведена итерационная схема

$$x_{n+1} = x_n - \alpha (\nabla^2 f(P y_0))^{-1} \nabla f(x_n)$$

Положив $\alpha = 1$, получим классический метод Ньютона:

$$x_{n+1} = x_n - (\nabla^2 f(P y_0))^{-1} \nabla f(x_n)$$

Вопрос: Как правильно вычислять $\left(\nabla^2 f(x_n)\right)^{-1} \nabla f(x_n)$?

Для данного выбора α удается доказать квадратичную скорость сходимости. Однако, метод сходится не из любого начального приближения x_0.

Теорема (локальная сходимость метода Ньютона): Пусть $f(x) \in C^2(\mathbb{R}^n)$, μ-сильно выпукла и $\nabla^2 f(x)$ --- L-Липшицев. Тогда для любого начального приближения x_0 из некоторой окрестности точки глобального минимума x_* метод Ньютона сходится с квадратичной скоростью.

Пример:

$$\varphi(t) = \sqrt{1+t^2} \to \min$$

Минимум достигается при $t = 0$. Запишем шаг метода Ньютона для данной функции:

$$\varphi'(t) = \frac{t}{\sqrt{1+t^2}}, \quad \varphi''(t) = (1+t^2)^{-\frac{3}{2}}$$
$$t_{n+1} = t_n - \frac{\varphi'(t_n)}{\varphi''(t_n)} = t_n - t_n(1+t_n)^2 = -t_n^3$$

Если $|t_0| >= 1$, то $|t_n| \geq |t_0| \geq 1$. Таким образом, метод не сходится к решению $t = 0$. Если $|t_0| < 1$, то метод сходится.

Демпфированный метод Ньютона, line search

В силу сильной выпуклости $f(x)$ оператор $\nabla^2 f(x) > 0$, поэтому направление $d_n = \left(\nabla^2 f(x_n)\right)^{-1} \nabla f(x_n)$ является направлением спуска. Поэтому:

$$\exists \lambda_n > 0: \quad f(x_n - \lambda_n d_n) < f(x_n)$$

В методе градиентного спуска существует оптимальный константный шаг $\lambda_n = \lambda = \frac{1}{L}$. Однако, в более сложных методах (в том числе методе Ньютона) мы не можем выбрать оптимальный константный шаг. Также константный шаг плохо учитывает локальную кривизну функции, поэтому необходим метод подбора шага λ_n на каждой итерации.

Существуют различные методы выбора λ_n:

1. Метод скорейшего спуска
2. Trust region
3. Неточная одномерная оптимизация

Мы рассмотрим только неточную одномерную оптимизацию. Рассмотрим функцию

$$\varphi(\lambda) = f(x_n - \lambda\, d_n)$$

В мы хотим, чтобы на каждом шаге функционал $f(x)$ "хотя бы немного уменьшался". Будем говорить, что λ удовлетворяет условию Армихо с константой $c \in (0, 1)$, если:

$$\varphi(\lambda) \leq \varphi(0) + c\lambda\varphi'(0)$$

Такой выбор λ гарантирует уменьшение значения функционала $f(x)$.

Для поиска λ, удовлетворяющего условию Армихо, можно использовать процедуру **бэктрекинга**:

1. Инициализируем $\lambda_0 \in (0, 1)$
2. Пока не выполнено правило Армихо:

1. $\lambda_{k+1} = \rho\lambda_k$
2. Если $\lambda_{k+1} < \varepsilon$, то поиск не удался

Демпфированный метод Ньютона

1. Инициализируем $x_0 \in \mathbb{R}^n$
2. Повторять до сходимости:
    1. Вычислить $d_n = \left(\nabla^2 f(x_n)\right)^{-1} \nabla f(x_n)$
    2. Если на предыдущем шаге $\lambda_{n-1} < 1$, то λ_n находится с помощью процедуры бэктрекинга. Если на предыдущем шаге $\lambda_{n-1} = 1$, то $\lambda_n = 1$
    3. $x_{n+1} = x_n - \lambda_n d_n$

Вычисление шага d_n, Newton-CG

Метод Ньютона требует вычисления выражения $d_n = \left(\nabla^2 f(x_n)\right)^{-1} \nabla f(x_n)$ на каждой итерации. В данном разделе будут обсуждаться методы вычисления шага d_n.

Решение СЛАУ $\nabla^2 f(x_n)\, d_n = \nabla f(x_n)$

Для вычисления d_n не требуется обращать матрицу $\nabla^2 f(x_n)$. Достаточно решить СЛАУ $\nabla^2 f(x_n)\, d_n = \nabla f(x_n)$. Для решения данной системы могут использоваться как прямые, так и итерационные методы.

Важно помнить, что при "плохо обусловленном" ландшафте функционала $f(x)$ матрица $\nabla^2 f(x_n)$ будет плохо обусловленной. Поэтому важно выбирать численно устойчивые алгоритмы для решения СЛАУ $\nabla^2 f(x_n)\, d_n = \nabla f(x_n)$.

Также при решении данного уравнения может использоваться изученная ранее техника предобуславливания.

Неточное решение СЛАУ $\nabla^2 f(x_n)\, d_n = \nabla f(x_n)$

В силу обратной устойчивости задачи (см. слайд 26) неточное решение СЛАУ $\nabla^2 f(x_n)\, d_n = \nabla f(x_n)$ эквивалентно точному решению другой СЛАУ $\tilde{H}_n d_n = \nabla f(x_n)$. Где $\tilde{H}_n$ "близка" к $\nabla^2 f(x_n)$. Можно получить, что при "достаточно точном" решении исходной СЛАУ матрица $\tilde{H}_n > 0$, поэтому полученное направление d_n останется направлением спуска. Поэтому при правильном выборе шага α_n (например, с использованием бэктрекинга) метод будет сходиться и при неточном вычислении d_n.

Чем точнее вычисляется d_n, тем быстрее будет сходиться метод в пересчёте на одну итерацию. Однако, более точное вычисление d_n может оказаться более трудоемким.

Поэтому важно соблюдать баланс между точностью вычисления d_n и сложностью одной итерации.

Метод сопряженных градиентов, Newton-CG

В вычислительной линейной алгебре существуют методы решения СЛАУ, не требующие хранения в памяти матрицы правой части. Рассмотрим метод сопряженных градиентов для решения СЛАУ $Ax = b$:

$$
\begin{aligned}
&r_1 = p_1 = b - Ax_0 \\
&\alpha_k = \frac{\langle r_k, r_k \rangle}{\langle Ap_k, p_k \rangle} \\
&x_{k+1} = x_k + \alpha p_k \\
&r_{k+1} = r_k - \alpha A p_k \\
&\beta_k = \frac{\langle r_{k+1}, r_{k+1} \rangle}{\langle r_k, r_k \rangle} \\
&p_{k+1} = r_{k+1} + \beta_k p_k
\end{aligned}
$$

Можно заметить, что для вычисления шагов $x_1, x_2, \ldots$ требуется уметь только умножать матрицу A на произвольный вектор. Это позволяет:

1. Вычислять Ax лениво, не храня при этом всю матрицу A
2. Использовать более широкий спектр матричных разложений для A

Вернемся к решению СЛАУ $\nabla^2 f(x_n)\, d_n = \nabla f(x_n)$. Попробуем проанализировать оба преимущества метода сопряженных градиентов:

1. При использовании автоматического дифференцирования существуют эффективные методы вычисления $\nabla^2 f(x_n)\, d_n$. Также ленивые вычисления позволяют экономить память (не нужно хранить всю матрицу $\nabla^2 f(x_n)$)
2. Для некоторых задач достаточно просто получаются матричные разложения, упрощающие вычисление $\nabla^2 f(x_n) d_n$

Пример: Рассмотрим логистическую регрессию. Нам дана выборка $\{x_i, y_i\}_{i=1}^l$, тогда оптимизируемый функционал будет иметь вид:

$$
f(w) = \sum_{i=1}^{l} \mathcal{L}(x_i, y_i, w), \quad \mathcal{L}(x, y, w) = -\log \sigma(y \langle x, w \rangle), \quad x_i, w \in \mathbb{R}^d
$$

Обозначим

$$
\begin{aligned}
&X = [x_1, \ldots, x_l] \in \mathbb{R}^{l \times d} \\
&s = (\sigma(y_1 \langle w, x_1 \rangle), \ldots, \sigma(y_l \langle w, x_l \rangle))^T \in \mathbb{R}^l \\
&B = \operatorname{diag}(s \odot (\mathbb{1} - s)) \in \mathbb{R}^{l \times l}
\end{aligned}
$$

В принятых обозначениях гессиан $\nabla^2 f(w)$ будет иметь вид:

$$\nabla^2 f(w) = X^T B X$$

Матрица X не зависит от весов w, поэтому для "обновления" гессиана достаточно посчитать только вектор s. Для умножения гессиана $\nabla^2 f(w)$ на вектор h может использоваться следующая схема:

1. Вычисляем $z_1 = X h$. Стоимость $O(l\,d)$
2. Вычисляем $z_2 = s \odot (1 - s) \odot z_1$. Стоимость $O(l)$
3. Вычисляем $\nabla^2 f(w) h = X^T z_2$. Стоимость $O(l\,d)$

Получаем:

1. Стоимость обновления $\nabla^2 f(w)$: $O(l)$
2. Стоимость вычисления $\nabla^2 f(w)\,h$: $O(l\,d)$
3. Дополнительная память: $O(l + d)$

При вычислении $\nabla^2 f(w) h$ без учета структуры задачи получилось бы:

1. Стоимость обновления $\nabla^2 f(w)$: $O(l\,d^2)$
2. Стоимость вычисления $\nabla^2 f(w)\,h$: $O(d^2)$
3. Дополнительная память: $O(d^2)$

Предположим, что $l >> d$, тогда использование структуры задачи позволяет получить выигрыш по памяти ($O(l + d)$ вместо $O(d^2)$) и выигрыш по скорости вычисления ($O(l\,d)$ вместо $O(l\,d^2)$)

Квазиньютоновские методы

Идея квазиньютоновских методов заключается в попытке "оценить кривизну" функционала $f(x)$ без использования оракула 2го порядка. Мы хотим научиться "приближать" обратный гессиан $H_n \approx (\nabla^2 f(x_n))^{-1}$. И вместо шага метода Ньютона $x_{n+1} = x_n - \alpha(\nabla^2 f(x_n))^{-1}\,\nabla f(x_n)$ делать шаг $x_{n+1} = x_n - \alpha H_n\,\nabla f(x_n)$.

Как было показано ранее, матрица H_n обязана быть положительно определенной. Далее будут рассмотрены методы выбора H_n.

Уравнение секущей (Secant equation)

Разложим $\nabla f(x)$ в полином Тейлора в окрестности точки x_n:

$$\nabla f(x) = \nabla f(x_n) + \nabla^2 f(x_n)\,(x - x_n) + \overline{o}(\|x - x_n\|)$$

Мы уже вычислили $\nabla f(x_{n-1}), \nabla f(x_n)$, подставим $x = x_{n-1}$ в полином:

$$\nabla f(x_{n-1}) = \nabla f(x_n) + \nabla^2 f(x_n)\,(x_{n-1} - x_n) + \overline{o}(\|x_{n-1} - x_n\|)$$

Нас интересует не сам гессиан, а обратная к нему матрица:

$$\nabla^2 f(x_n)(\nabla f(x_n) - \nabla f(x_{n-1})) = (x_n - x_{n-1}) \cdot (1 + \overline{o}(1))$$

В данном уравнении мы не знаем только гессиан $\nabla^2 f(x_n)$. Потребуем, чтобы приближение H_n удовлетворяло **уравнению секущей**:

$$H_n(\nabla f(x_n) - \nabla f(x_{n-1})) = (x_n - x_{n-1})$$

В одномерных пространствах данное уравнение однозначно задает скаляр H_n. На данном приближении основан метод секущих для поиска корней уравнений.

Если размерность пространства ≥ 2, то уравнение секущей не позволяет однозначно определить матрицу H_n, поэтому существуют различные методы выбора H_n.

Замечание: Уравнение секущей не всегда имеет хоть одно решение в классе $H_n > 0$. Умножим левую и правую часть уравнения на $(\nabla f(x_n) - \nabla f(x_{n-1}))^T$:

$$(\nabla f(x_n) - \nabla f(x_{n-1}))^T\, H_n\, (\nabla f(x_n) - \nabla f(x_{n-1})) = (\nabla f(x_n) - \nabla f(x_{n-1}))^T\, (x_n - x_{n-1})$$

В силу $H_n > 0$ получим, что левая часть > 0. Тогда необходимым условием существования решения является:

$$(\nabla f(x_n) - \nabla f(x_{n-1}))^T\, (x_n - x_{n-1}) > 0$$

Замечание: Гессиан $\nabla^2 f(x_n)$ может не удовлетворять уравнению секущей, так как уравнение секущей было получено из точного равенства для гессиана путем отбрасывания $\overline{o}(1)$.

Метод Symmetric Rank-one (SR1)

Инициализируем матрицу $H_0 = I_d$. На каждом шаге процесса оптимизации будем корректировать матрицу $H_{n-1} \to H_n$ так, чтобы она удовлетворяла уравнению секущей в новой точке. Так как уравнение секущей задает "поведение" H_n лишь в направлении $\nabla f(x_n) - \nabla f(x_{n-1})$, то для выполнения уравнения секущей будет достаточно изменить матрицу H_n на одномерном подпространстве. Отсюда следует, что для выполнения уравнения секущей достаточно сделать обновление H_n при помощи добавки ранга 1:

$$H_n = H_{n-1} + u_n\, u_n^T$$

Подставив данное выражение в уравнение секущей получим:

$$H_n = H_{n-1} + \frac{(s_n - H_n y_n)(s_n - H_n y_n)^T}{(s_n - H_n y_n)^T y_n}$$

где:

$$
\begin{aligned}
s_n &= x_n - x_{n-1} \\
y_n &= \nabla f(x_n) - \nabla f(x_{n-1})
\end{aligned}
$$

Стоимость обновления матрицы H_n:

1. Стоимость вычисления $(s_n - H_n y_n)(s_n - H_n y_n)^T$ --- $O(d^2)$
2. Стоимость вычисления $(s_n - H_n y_n)^T y_n$ --- $O(d)$
3. Стоимость сложения: $O(d^2)$

Замечание: Метод имеет существенный недостаток. Для положительной определенности H_n требуется неотрицательная определенность $\frac{(s_n - H_n y_n)(s_n - H_n y_n)^T}{(s_n - H_n y_n)^T y_n}$. Матрица $(s_n - H_n y_n)(s_n - H_n y_n)^T \geq 0$. Тогда как скаляр $(s_n - H_n y_n)^T y_n$ может оказаться < 0. В данном случае приходится придумывать правила "пропуска" обновления H_n.

Метод Бройдена–Флетчера–Гольдфарба–Шанно (BFGS)

Как было показано ранее, в схеме обновления SR1 могут возникать проблемы при $(s_n - H_n y_n)^T y_n < 0$. В методе BFGS предлагается выбрать среди всех матриц удовлетворяющих уравнению секущей матриц $H > 0$ матрицу H_n, наиболее "близкую" к H_{n-1}

Мы хотим оптимизировать некоторый функционал близости к H_{n-1} на множестве $D = \{H \mid H > 0,\ Hy_n = s_n\}$. Получаем задачу условной оптимизации, причем одно из ограничений имеет вид $H > 0$. Перейдем к параметризации матриц $H = JJ^T$. Любую матрицу $H > 0$ можно представить в таком виде и любая матрица $JJ^T > 0$.

Тогда получим $D = \{J\,J^T \mid J\,J^T y_n = s_n\}$. Получаем нелинейное ограничение на фактор J. Перейдем от нелинейного ограничения к двум линейным:

$$
J\,J^T y_n = s_n \iff (u = J^T y_n) \ \wedge \ (Ju = s_n)
$$

Так как изначально не был зафиксирован функционал близости H к H_{n-1}, то на данном этапе мы можем выбрать его удобным для себя образом:

$$
\begin{aligned}
\min_{J,u} \quad & \|J - J_{n-1}\|_F^2 \\
\text{s.t.} \quad & J^T y_n = u \\
& Ju = s_n
\end{aligned}
$$

Получаем задачу квадратичного программирования с ограничениями типа равенств. Данная задача имеет аналитическое решение. Подставив решение J_* в $H_n = J_*\,J_*^T$ получим:

$$H_n = \left(I - \rho_n s_n y_n^T\right) H_{n-1} \left(I - \rho_n s_n y_n^T\right) + \rho_n s_n y_n^T, \text{ где } \quad \rho_n = \frac{1}{\langle y_n, s_n \rangle} > 0$$

Вопрос: Единственна ли параметризация $H = J\,J^T$. Если нет, то зависит ли результат от выбора параметризации?

Метод Limited Memory BFGS

Недостатком метода BFGS является необходимость хранить матрицу H_n в памяти. Предположим, что мы уже умеем считать $H_{n-1}\,h$ для любого вектора h. Рассмотрим схему вычисления $H_n\,h$:

1. Вычисляем $z_0 = \rho_n s_n \langle y_n, h \rangle$
2. Вычисляем $z_1 = h - z_0$
3. Так как мы уже умеем умножать H_{n-1} на любой вектор, то вычисляем $z_2 = H_{n-1} z_1$
4. Вычисляем $z_3 = z_2 - \rho_n s_n \langle y_n, z_2$
5. Получаем $H_n h = z_3 + z_0$

Вместо хранения матриц H_n будем хранить векторы $s_{k-1}, \ldots, s_{k-m}$ и $y_{k-1}, \ldots, y_{k-m}$. Где m некоторое небольшое целое число, обычно $m = 10$. Тогда рассмотрим схему вычисления $H_n \nabla f(x_n)$ по методу L-BFGS:

1. Инициализируем $H_{n-m} = I_d$
2. Рекурсивно вычисляем $H_n \nabla f(x_n)$ по описанной выше схеме.

Сложность L-BFGS:

- Память: $O(m\,d)$
- Арифметические операции: $O(md)$